Technical Manifesto: Scientific Software Development

A Handbook for Scientific Workflows in the Era of Vibecoding

W. H. W. Thompson

2026-01-27

Table of contents

1 The Vision **2**
1.1 Expectation vs. Reality 2

2 Without SLURM We Would Have Cured Cancer By Now **2**

3 The Iron Rule of Scientific Software Development **2**
3.1 Opportunity Cost 2

4 The Scientific Software Checklist **2**

5 1. The Foundation **3**
5.1 Python Features 3
5.2 Environment Management (UV) 3
5.3 Acceleration (JAX / PyTorch) 3
5.4 Validation (Pydantic) 4

6 2. The Plumbing **4**
6.1 Command Abstraction (Just) 4
6.2 Orchestration (Snakemake) 4
6.3 Testing (Pytest) & CI (GitHub Actions) 5
6.4 Tracking & Logs (WandB) 5
6.5 Data Storage (DuckDB) 6

7 3. Resources & AI Integration **6**
7.1 AI Tools 6
7.2 Automation & MCP 6
7.3 Workflow Inspiration 7

1 The Vision

1.1 Expectation vs. Reality

In scientific research, we often imagine ourselves spending most of our time thinking deeply and understanding complex systems. However, the reality of computational research is often dominated by "plumbing": managing environments, fighting with SLURM, and debugging boilerplate code.

The goal of this manifesto is to **automate the plumbing** so you can focus on the **science**.

2 Without SLURM We Would Have Cured Cancer By Now

3 The Iron Rule of Scientific Software Development

1. If you take the time to build out code the right way, you will never use it.
2. If you write sloppy code, it will bite you in the ass.
3. **Damned if you do. Damned if you don't.**

3.1 Opportunity Cost

Investing in modern development practices might feel slow initially, but it prevents the "Conda Hell" and "Terminal Staring" that consume the majority of research time.

4 The Scientific Software Checklist

Component	Recommended Tool
Reproducible Environment	**UV** package manager & lockfiles
Data Validation	**Pydantic** schemas & types
Hardware Acceleration	**PyTorch** and **JAX**
Code Reliability	**Pytest** and **CI/CD**
Workflow Automation	**Snakemake** pipelines
Experiment Tracking	**Weights and Biases**
Publication/Dashboards	**Quarto**

5 1. The Foundation

5.1 Python Features

Keep logic in `.py` files rather than notebooks to ensure it is portable and testable. Use type annotations to catch bugs before they reach the cluster.

```python
# Type-safe scientific logic
def simulate_trajectories(
    n_agents: int,
    time_steps: int,
    params: dict[str, float]
) -> list[np.ndarray]:
    """Simulates N trajectories over T steps."""
    ...
```

5.2 Environment Management (UV)

UV is a modern replacement for Conda that is significantly faster and more reliable.

```toml
# pyproject.toml
[project]
name = "scientific_dev"
requires-python = ">=3.12"
dependencies = [
    "jax>=0.9.0",
    "snakemake>=8.4.0",
    "pydantic>=2.12.5",
]
```

5.3 Acceleration (JAX / PyTorch)

If a computation can be vectorized, it should be offloaded to a GPU. JAX's `vmap` is particularly powerful for running thousands of simulations in parallel.

```python
import jax.numpy as jnp
from jax import vmap

def compute_score(x):
    return jnp.sum(jnp.sin(x) ** 2)
```

```
batch_score = vmap(compute_score)
scores = batch_score(large_input_matrix)
```

5.4 Validation (Pydantic)

Enforce structure on your parameters and results using Pydantic. This makes your code more robust and easier for AI agents to reason about.

```
class NLPModelConfig(BaseModel):
    model_name: str = Field(default="distilbert-base-uncased")
    adapter_dim: int = Field(default=256)

    @field_validator('adapter_dim')
    def validate_adapter_dim(cls, v: int):
        if v <= 0:
            raise ValueError("adapter_dim must be positive")
        return v
```

View Schema

6 2. The Plumbing

6.1 Command Abstraction (Just)

Use a Justfile to hide complex commands behind simple aliases. This ensures reproducibility across the team.

```
# Run full pipeline
all p="local" c="configs/nlp_baseline.yaml":
    uv run snakemake --workflow-profile ./workflow/profiles/{{p}} --configfile {{c}}
```

View Justfile

6.2 Orchestration (Snakemake)

Snakemake manages complex dependency graphs and integrates seamlessly with Slurm for VACC execution.

```
rule train_bert_classifier:
    output:
        weights = "results/{cfg_name}/{params_path}/model.pt",
        stats = "results/{cfg_name}/{params_path}/training_stats.parquet"
    resources:
        gpu=1
    shell:
        "PYTHONPATH=src python scripts/run_training.py "
        "--config configs/{wildcards.cfg_name}.yaml "
        "--output {output.weights}"
```

View Full Rule

6.3 Testing (Pytest) & CI (GitHub Actions)

Validate research code quality and ensure reproducibility on every push.

```
# tests/test_nlp_logic.py
def test_validation():
    with pytest.raises(ValueError):
        NLPModelConfig(adapter_dim=-1)
```

View CI Workflow

6.4 Tracking & Logs (WandB)

Weights and Biases provides real-time tracking of experiments and ensures every run is versioned.

```
wandb.log({
    "train/step_loss": loss.item(),
    "train/learning_rate": self.optimizer.param_groups[0]["lr"],
    "train/global_step": epoch * len(self.train_loader) + step
})
```

View Trainer

6.5 Data Storage (DuckDB)

Use DuckDB to query Parquet files using standard SQL.

```
SELECT s.*, m."test/auroc"
FROM read_parquet('results/**/training_stats.parquet') s
LEFT JOIN read_parquet('results/**/test_metrics.parquet') m
  ON s.full_run_name = m.full_run_name
```

7 3. Resources & AI Integration

7.1 AI Tools

- **IDEs**: VSCode and Cursor are the gold standard for AI-assisted research.
- **LLMs**: Gemini Pro and Claude provide high-context reasoning for complex math and code.

7.2 Automation & MCP

Use the Model Context Protocol to bridge your research library (Zotero, NotebookLM) directly into your AI agent's reasoning.

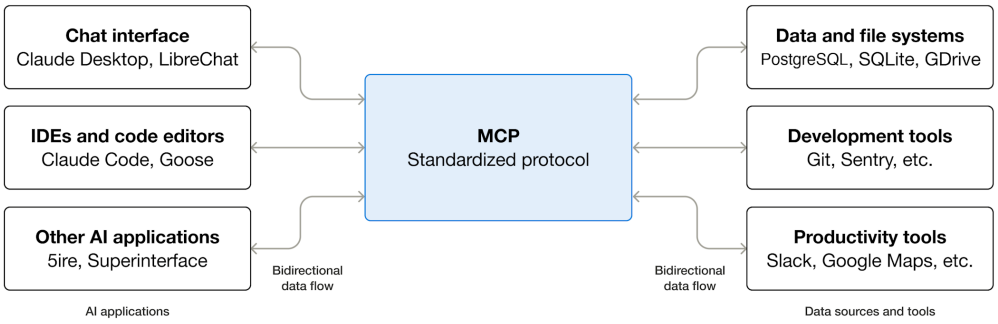

Figure 1: MCP Architecture

7.3 Workflow Inspiration

For clean pipeline design and project organization, refer to Fitz's Workflow.